



University of Glasgow | School of
Computing Science

Honours Individual Project Dissertation

PRIVACY-PRESERVING PERSONAL DATA MANAGEMENT UTILISING BLOCKCHAIN TECHNOLOGY AND UMBRAL PROXY RE-ENCRYPTION

Xiaozu Hu

March 27 2026

Abstract

Centralised file sharing services require users to delegate control of their data to third-party providers who can read, share, or restrict access to it without recourse. This project investigated whether a combination of Umbral proxy re-encryption, Ethereum smart contracts, and the InterPlanetary File System could provide anonymous, decentralised file transfer in which no intermediary — including the proxy facilitating re-encryption — gains access to the plaintext. A protocol was designed in which an owner encrypts a file and stores it on IPFS, a recipient registers their public key on-chain, and a proxy selected performs a cryptographic transformation on the ciphertext capsule, enabling the recipient to decrypt using only their own private key. All coordination is mediated by a smart contract that enforces the correct ordering of operations and prevents unauthorised parties from interfering with the transfer. A prototype was implemented in Python using the nucypher-core Umbral library, Solidity smart contracts deployed on a local Ethereum testnet via Foundry. End-to-end evaluation confirmed correct operation across all protocol stages. Known limitations of the prototype include unencrypted key fragments, a single-proxy configuration that does not exercise the full Umbral threshold property, and the pseudonymous rather than anonymous nature of Ethereum addresses. The project demonstrates the feasibility of cryptographically enforced, intermediary-free file transfer, and should serve as a foundation for further research in the subject area

Acknowledgements

Education Use Consent

Consent for educational reuse withheld. Do not distribute.

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aim of Project	2
1.3	Scope of Project	2
2	Background	3
2.1	Proxy Re-encryption	3
2.1.1	Traditional PRE	3
2.1.2	Umbral PRE	4
2.2	Smart Contracts	4
2.3	Decentralised Storage - IPFS	4
2.4	Related Works	5
2.4.1	Decentranet: An Ethereum, Proxy Re-Encryption and IPFS Based De-centralized Internet	5
2.4.2	A Proxy Re-Encryption Approach to Secure Data Sharing in the Internet of Things Based on Blockchain	6
3	Analysis/Requirements	8
3.1	Problem Statement	8
3.2	Requirements Gathering	8
3.3	Threat Modelling	9
4	Design	10
4.1	System Architecture	10
4.2	Cryptographic Design	10
4.2.1	Set-Up Phase	10
4.2.2	Kfrag Generation Phase	11
4.2.3	Re-Encryption Phase	12
4.2.4	Decryption Phase	12
4.3	Data Flow	13
5	Implementation	14
5.1	Technology Stack	14
5.2	Technical Decisions	14
5.2.1	Segmentation of Scripts to Simulate Distributed Roles on a Single-Machine	14
5.2.2	Supplementing JSON Files for IPFS Upload and Retrieval Calls	15
5.3	Smart Contract - Policy Registry Implementation	15
5.3.1	Policy Registry	15

5.4	Proxy Functions	16
5.5	Owner and Recipient Implementation	17
5.5.1	Data Owner	17
5.5.2	Recipient	17
5.6	Implementation Limitations	18
5.6.1	Unencrypted kfrag Security Limitation	18
5.6.2	Blockchain Pseudonymity Security Limitation	18
6	Conclusion	19
6.1	Summary	19
	Appendices	20
A	Appendices	20
	Bibliography	22

1 | Introduction

Over the years, the increased importance and usage of cloud-based services have fundamentally changed how individuals store and share their personal data. Services offered by providers such as Google, YouTube or Facebook have become deeply embedded in a persons daily life. They offer access to storage, communication, and sharing of media at no mandatory monetary cost. However, this convenience conceals a significant transfer of control that goes largely unexamined and unnoticed by their users. By accepting the terms and conditions of these service providers, users essentially sign away ownership and control over their personal data. The data which is stored on the providers centralised servers are governed by their policies, under their control, which makes the data potentially accessible to the provider, their partners, and authorities that compels disclosure.

This relinquishing of rights presents a fundamental disparate between legal ownership and practical control over personal data. A user who uploads a file to a cloud storage service retains legal ownership under data protection laws, but has no practical method of enforcing their ownership. The problem is further pronounced when files are shared between two users. Current mechanisms for file transfer typically require both users to trust a common intermediary service with both the content of the transfer and the metadata revealing that the transfer took place.

This project proposes an alternative protocol that addresses this problem by returning the cryptographic control of personal data back to the individual. Using a combination of Umbral Proxy Re-encryption(PRE), Ethereum smart contracts, and the InterPlanetary File System(IPFS), the proposed system enables a data owner to transfer a file to a designated recipient such that no intermediary can access the file contents. All coordination in this system is managed by immutable smart contracts on a public blockchain, providing a tamper-evident audit trail without requiring any trusted central authority.

1.1 Motivation

The motivation for this project comes from a widely observed but under-addressed issue in modern computing, the gap between data ownership as a legal right and data ownership as a technical reality. While there are legal statues in place such as the General Data Protection Regulation (GDPR) that protects an individuals rights over their personal data, the technical architecture of centralised cloud services makes those rights difficult to enforce practically. A user is unable to know what a service provider does with their data, unable to prevent the provider, its subsidiaries, and other authorities from accessing their data, and unable to guarantee that the data is completely deleted when requested. This lack of control carries significant risk for certain types of users that frequently handles privileged data like journalists as well as legal and medical professionals.

Blockchain technology and cryptographic access control methods offer a way of resolving this issue. A blockchain provides a decentralized, tamper evident coordination layer that eliminates the need of a trusted central authority. Umbral PRE allows designated proxies to act as an intermediary to transfer a cipher-text from being decipherable using one private key to another without exposing the underlying plaintext, enabling secure delegation of decryption rights.

Combining these two technologies together with a decentralized storage network like IPFS as the storage host creates a system in which the owner of data can share it selectively, without needing to delegate control over the data to any third party.

This project recognizes that existing work in this area does exist, but tends to lean towards theoretical proposals or systems that are impractical to implement due to scalability issues. This project produces a focused prototype implementation that clearly demonstrates the interactions between Umbral PRE, blockchain smart contracts and the decentralised storage server, which provides a basis for future work and further development in the subject area.

1.2 Aim of Project

The primary aim of this project is to design and implement a decentralised cryptographic file transfer protocol that returns control over data back to the data owner. The protocol should require no trusted third party, should protect against proxies that attempt to learn file contents, should have no direct interactions between the data owner and the recipient, and should use a blockchain for coordination.

The aims are as follows:

- To design a protocol for anonymous file transfer that utilises Umbral PRE with Ethereum smart contracts and IPFS, such that no single party other than the owner and the intended recipient can access the plaintext.
- To implement a prototype of the protocol where the owner, recipient, and proxy roles are individual script components that interact only through smart contracts and IPFS.
- To design and implement smart contracts that manages the registration and coordination of the participants on-chain
- To evaluate the prototype with respect to the gas cost, correctness, and fidelity with respect to the proposed design.
- To identify the security properties the prototype contains and its limitations.

1.3 Scope of Project

The scope of this project is constrained for focused research of the core cryptographic and coordination mechanisms. The developed prototype is deployed on a local Ethereum test-net, Sepolia, using Anvil, a Ethereum node simulator in the Foundry development toolkit. This decision was made so that gas cost measurements can be made without real economic costs.

This project was developed and evaluated in a single-proxy configuration, meaning only one proxy is used for the re-encryption process each time. This is a simplification of the full Umbral threshold PRE design, in which a threshold of multiple proxies should cooperate and no singular proxy is able to learn any information. The single-proxy design was chosen as it reduces the complexity of the implementation without affecting the correctness of the underlying cryptographic operations of Umbral PRE. The design and implementation can also naturally be extended to be operable with multiple proxies since the threshold property is still implemented, just not utilised.

The project does not implement attribute-based encryption or zero-knowledge proof techniques, both of which were identified in the initial project specification as potential components. These changes were discussed with the project supervisor and approved to be further explored in the future. Discussions of these techniques will be included in the background section as related work.

2 | Background

This chapter introduces the core technologies utilised in the design of the protocol: PRE, Umbral threshold PRE, Smart contracts and IPFS. This chapter will also examine existing work with similar systems and identify the gaps in their design.

2.1 Proxy Re-encryption

2.1.1 Traditional PRE

Traditional PRE schemes can be seen as a delegation of decryption rights where the delegator, in our case the data owner, gives the right to decrypt a ciphertext to the delegatee, which would be the recipient. It achieves this using a singular proxy that re-encrypts the ciphertext with the delegators public key such that the delegatee can decrypt the file using their private key. The proxy should also not be able to learn any information about the ciphertext during this process. The security of PRE schemes is usually defined under two security notions: *indistinguishability against chosen-plaintext attacks (IND-CPA)* and *indistinguishability against chosen-ciphertext attacks (IND-CCA)*. Both notions are formalised as reductions to hard problems like the Computational and Decisional Diffie-Hellman problems, capturing the inability of an adversary to distinguish ciphertexts for known messages (Nuñez et al., 2017).

A general traditional PRE scheme can be formally defined by the following tuple of algorithms (adapted from Singh et al., 2014):

$KeyGeneration(n) \rightarrow (pk_a, sk_a)$

On input a security parameter n , this algorithm outputs a pair of public and secret keys for the user

$Encrypt(pk, M) \rightarrow C_i$

This algorithm takes input as public parameters, a public key pk and a message M , and outputs a ciphertext C_i

$ReEncryptionKey(sk_i, pk_i, pk_j) \rightarrow rk_{i,j}$

This algorithm takes as inputs, a secret key sk_i , a public key pk_i , and a public key pk_j , and outputs a re-encryption key $rk_{i,j}$

$ReEncryption(rk_{i,j}, C_i) \rightarrow C_j$

On input a ciphertext C_i and re-encryption key $rk_{i,j}$, this algorithm outputs a re-encrypted ciphertext C_j

$Decrypt(sk_j, C_j) \rightarrow M$

This algorithm takes public parameters, a private key sk_j and a ciphertext C_j , and outputs message M

2.1.2 Umbral PRE

The Umbral PRE scheme, is a "threshold proxy-encryption scheme following a Key-Encapsulation Mechanism (KEM) approach" (Núñez, Aransay and González, 2017).

KEM is a method that is used between two parties to securely establish a shared secret key. The main function of KEM in the Umbral PRE scheme is to generate a symmetric secret key k and to produce a capsule C that encapsulates k using the public key of the delegator. The ciphertext c is produced by encrypting the plaintext using k . On re-encryption, only the capsule C is re-encrypted by the proxy to produce C' , where when the delegatee decapsulates C' using their private key, they can recover k and decrypt the ciphertext.

The main differences between Umbral PRE and traditional PRE are:

- Instead of using a singular proxy for the re-encryption process and giving it the re-encryption key, Umbral splits it into n key fragments (kfrags) using Shamir secret sharing, such that any t of n fragments are sufficient to perform re-encryption but fewer than t would reveal nothing about the key. The inclusion of the t threshold means that one proxy would not hold enough information to re-encrypt and at least t proxies must cooperate.
- The inclusion of KEM makes it such that the capsule KEM produces is the main object of encryption and re-encryption instead of the ciphertext in traditional PRE. This means that the ciphertext is never accessed by the proxies, only the capsule, making the protocol more efficient.

2.2 Smart Contracts

Smart contracts are self-executing programs deployed on a blockchain that process incoming transactions according to predetermined, deterministic algorithms. They can be interacted with when a user sends a transaction to the contract address, typically to call a specific function within the contract. The contract executes the function which either triggers a state change in the smart contract memory, trigger predetermined events, or trigger another transaction (De Filippi et al., 2021). Since execution of a smart contract function is deterministic and global, the outcome of any contract interaction is verifiable by all participants without a trusted central authority. It is also tamper evident and easily auditable since all transactions made on the blockchain are recorded in the ledger.

2.3 Decentralised Storage - IPFS

IPFS is a decentralised storage protocol that relies on peer-to-peer networks instead of centralised servers. A fundamental difference between IPFS and other traditional storage systems is that IPFS uses a content identifier (CID) to point to stored data. CIDs do not indicate where the data is stored, but rather forms an address based on the content of the data itself (Protocol Labs, 2024). The length of a CID is also independent from the size of the data being stored, which means larger data sets will not suffer from longer retrieval times on a fetch attempt. A pitfall that IPFS suffers from is that file accessibility is directly impacted by peer accessibility, meaning that if node that actively holds the data is not online, the data becomes unreachable (Shi et al., 2024). The protocol does include a pinning function that allows a user to always keep the data stored somewhere, the default being the users local node, which is utilised in the project to reinforce data accessibility.

2.4 Related Works

2.4.1 Decentranet: An Ethereum, Proxy Re-Encryption and IPFS Based Decentralized Internet

The Decentranet, proposed by Koushik Bhargave Muthe, et al in a 2020 paper is similarly motivated by the fact that third party cloud services have been given more control and power over personal data, leading to problems like lack of privacy and resiliency (Muthe et al., 2020). The proposed decentranet protocol, is a fully decentralised internet that resolves this issue by utilising blockchain technology, PRE, and ZKP, to connect users on the internet directly without the need for centralised entities while keeping the data secure.

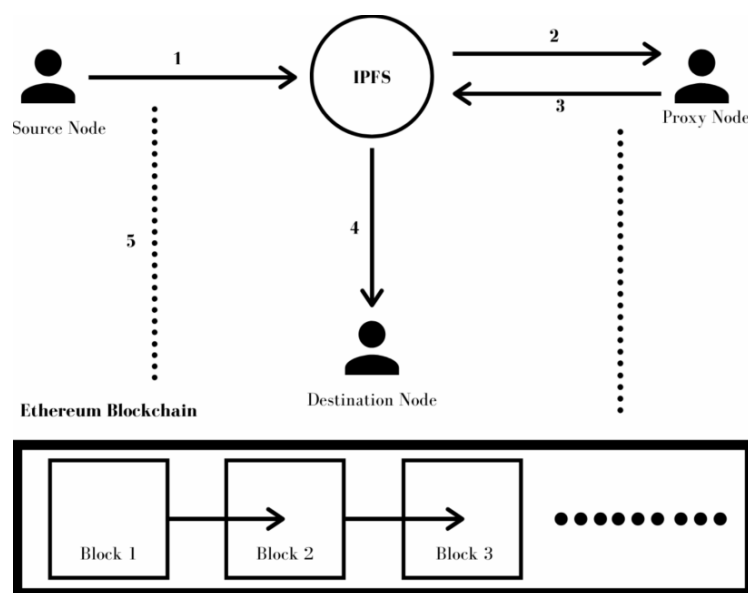


Figure 2.1: Proposed Architecture of The Decentranet. Source: (Muthe et al., 2020)

The proposed architecture is visualised in the above diagram and its steps are described as follows (Muthe et al., 2020):

- The Source Node(data owner) uploads the encrypted file to the IPFS network containing the targeted destination address list.
- The Proxies in the network take up the encrypted file based on the Proof of Stake mechanism
- The Proxies re-encrypt the file to the prime destinations and upload it back to the IPFS
- The destination node receives the re-encrypted file and decrypts it using its secret key.
- Every transaction in the IPFS network is recorded in the Ethereum Blockchain leveraging the power of Smart Contracts

The proposed protocol was evaluated and found to be an improvement over using the traditional HTTPS protocol in terms of latency during transfers and consumption of computational power. However, a vulnerability was identified in the protocol, unlinkability for its users was not ensured. In the proposal it is stated that the proxies take up the responsibility of re-encryption based on a Proof of Stake mechanism, meaning proxy participation and reward transactions are recorded on the public blockchain ledger. Since blockchain transactions contain publicly accessible timestamps, an observer could potentially correlate the timing of re-encryption transactions with payment

transactions to derive relationships between users and proxies. This vulnerability is demonstrated by the TRAP attack (Wang et al., 2025), which exploits transaction timestamps to estimate transaction confirmation intervals and derive pseudonym associations between transactions processed in the same confirmation interval, effectively de-anonymising participants. The proposed protocol also acknowledges a scalability issue in terms of production due to restrictions in existing internet architecture (Muthe et al., 2020).

While the paper claims that non-interactive ZKP is utilised to add a layer of privacy by concealing node identities during blockchain transactions, which would foil TRAP attacks, this claim is not sufficiently supported. No description of how this concealment is constructed is provided; Hence, it is not possible to evaluate whether the claimed concealment of proxy identities is achievable in the described architecture. This lack of description means the unlinkability problem cannot be verified and should be treated as a possible vulnerability.

2.4.2 A Proxy Re-Encryption Approach to Secure Data Sharing in the Internet of Things Based on Blockchain

The proposed protocol by Kwame Opuni-Boachie..., et al aims to fulfill the primary goal of being secure against attacks that hinder IoT technologies from providing required services, in addition to those that pose threats to the confidentiality, integrity, and privacy of data. By leveraging blockchain technologies and combining PRE with identity-based encryption (IBE) and information-centric networking (ICN), the protocol aims to: enhance the security and privacy in data-sharing systems, ensure fine grained data access control, and promise a sufficient quality of service in data delivery. (Manzoor et al., 2021).

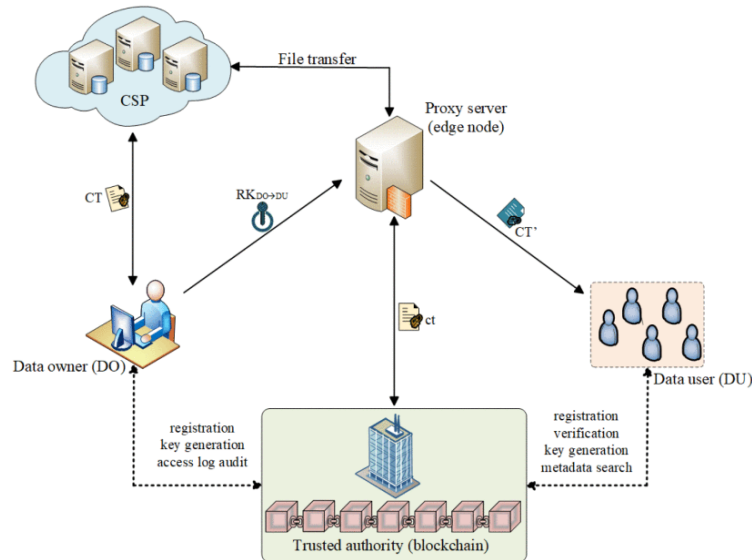


Figure 2.2: Proposed Architecture of the protocol. Source:

The proposed architecture is visualised in the above diagram. To summarise (Manzoor et al., 2021):

- Data owner encrypts the file and uploads the ciphertext to the cloud service provider (CSP)
- Metadata and access control information are stored on the blockchain
- When a data user requests access to the data, the owner generates a re-encryption key using the recipient's identity and sends it to the proxy

- The proxy then fetches the ciphertext from the CSP, applies the re-encryption key to transform it, and sends the re-encrypted ciphertext to the data user
- Finally, the recipient decrypts the re-encrypted ciphertext using their private key

While the proposed system achieves its stated aims of data confidentiality and fine-grained access control, it does not realise the principle of returning data ownership to the data owner. By outsourcing ciphertext to a centralised CSP, the scheme delegates some degree of control to the CSP. The CSP would retain the ability to restrict or deny access to data by going offline, blocking requests, or when directed by other authorities, all scenarios where the data owner would be unable to prevent through the blockchain alone. Replacing the CSP with a decentralised storage network like IPFS would resolve this issue, which this project does.

1 <https://ieeexplore.ieee.org/abstract/document/9225483> (blockchain + PRE + ZKP)

2 <https://ieeexplore.ieee.org/abstract/document/9442931> (blockchain + PRE)

3 | Analysis/Requirements

3.1 Problem Statement

The general goal of this project is to propose a protocol for conducting anonymous, controllable and monitored transferring of data from a data owner to a recipient, while having no central authority governance.

Existing approaches typically fall into one of two categories. They either rely on a trusted third party intermediary like a cloud storage provider, or still require the owner and recipient parties to discover and authenticate each other directly, often leaking information in the process.

This project utilises Umbral PRE as an alternative method of data transfer. In an Umbral PRE scheme, a proxy can transform a ciphertext encrypted by the owner, into one decryptable using the recipient's private key. It is able to achieve this without ever learning the underlying plaintext, or either party's private keys. The proxy is therefore a purely mechanical role, and the security of this scheme would not need to depend on whether the proxy is able to be trusted with secret data. The recipient also never learns of the data owner's private key, keeping both parties entirely anonymous from each other.

This project is deployed on the Ethereum blockchain to address the issue of coordination in a system without central governance. In a fully decentralised system, there is no straightforward mechanism for: the owner to register who the recipient is, the proxy to announce its participation, or the participants in the system to coordinate based on the progress of the data transfer. A blockchain smart contract provides a shared, secure registry with coordination functionality, utilising signals and events, that all parties can listen to without revealing themselves or their actions.

IPFS was chosen as the storage platform due to its decentralised nature which align with the project aims, and the efficiency of small sized CIDs.

3.2 Requirements Gathering

Requirements were gathered through a combination of listed requirements from the project supervisor, a review of related literature on current protocols of data transfer schemes, proxy re-encryption and blockchain based access control methods.

The identified functional requirements are as follows:

- The system should allow the data owner to encrypt a file and store it on a decentralised storage network without revealing the data to anyone else
- The system should allow an owner or recipient to register their public key on-chain without linking the key to any real world identity
- The owner should be able to delegate the decryption of the ciphertext to a recipient by generating re-encryption key fragments and distributing them to a registered proxy
- Proxies should be able to re-encrypt the capsule component of a ciphertext using their assigned key fragment without learning about the plaintext or the key

- The recipient should be able to retrieve re-encrypted capsule fragments, verify and combine them, then decrypt the file using only their own private key
- All coordination between the data owner and the recipient should be mediated using smart contracts on a Ethereum blockchain
- The recipient should be able to successfully request access to the data at any time as long as the owner has uploaded it to IPFS, even if the owner is offline
- The proxy should be able to successfully perform re-encryption without requiring the owner to be online

Some non-functional requirements were also identified and act as constraints on the system's design:

- The plaintext should be kept confidential and no party other than the owner and the correct recipient should be able to recover the plaintext.
- No data on-chain or on the decentralised data server should be able to reveal the identity of the owner or recipient.
- The intended recipient should be able to verify that the capsule fragments received are genuine (from the data owner) and have not been tampered with.
- The protocol should always complete successfully if the recipient and at least 1 registered proxy is online assuming the owner successfully uploaded the desired data to IPFS

3.3 Threat Modelling

The threat model assumes that the owner and recipient do not trust each other but act honestly, they follow the protocol but want to learn as much about each other as possible.

Each proxy does not trust anybody but acts honestly, but up to n of them may be compromised by an attacker.

The blockchain is assumed to be online and consistent, its transactions may be viewable by an attacker but its state cannot be changed by them.

The system does not attempt to hide the data transfer from an attacker since blockchain transactions are inherently public. Even though Ethereum wallet addresses are used in coordination effort, linking those addresses to real world identities is possible but out of scope for this project.

4 | Design

This chapter describes the process of designing the system. The design of the system is of an ideal production-ready system that uses multiple proxies with all interactions properly established in the blockchain smart contracts.

4.1 System Architecture

The system comprises five parties: the data owner, the recipient, a group of proxies, the blockchain, and IPFS. The data owner and recipient should never communicate directly and all coordination should be done through the blockchain using smart contracts and proxies.

The system design avoids any single point of failure. The smart contract is immutable once deployed onto the blockchain and cannot be modified by any participant. The group of proxies is available to any participant who registers their public key on-chain.

Segments of a sequence diagram will be shown to aid the explanation of the protocol phases. The entire diagram can be seen in A.1

4.2 Cryptographic Design

4.2.1 Set-Up Phase

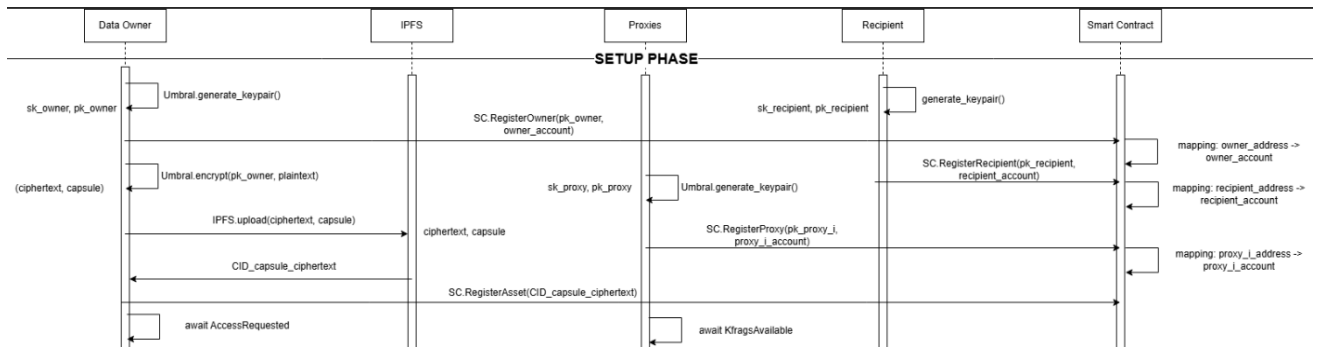


Figure 4.1: Screenshot of the set-up phase of the protocol in the sequence diagram

The data owner, recipient, and proxies all generate their key-pairs and registers their public keys on the blockchain using a smart contract, mapping their addresses to their public keys. The data owner then encrypts the plaintext with parameters being the plaintext and their public key, which outputs the ciphertext (encrypted using the generated symmetric key) and a capsule (the symmetric key encapsulated using the owner public key). The data owner then uploads the ciphertext and capsule to IPFS to get the associated CID identifier and registers the asset using a smart contract. Each registered asset is given an assetID which is their index in a list of all

registered assets stored in the smart contract registry. After the IPFS upload, the asset is pinned to ensure accessibility and the owner goes into an await state, listening for the *AccessRequested* event. The proxies also enter an await state, listening for the *KfragsAvailable* event.

4.2.2 Kfrag Generation Phase

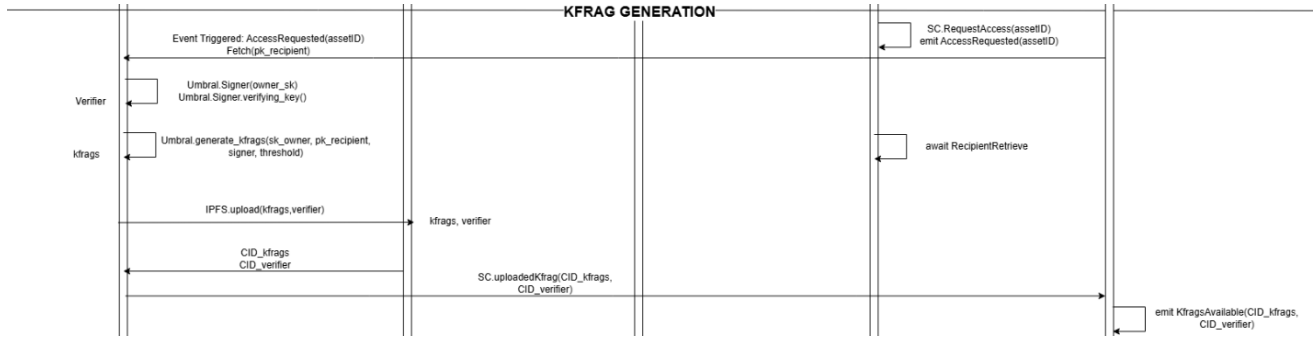


Figure 4.2: Screenshot of the kfrag generation phase of the protocol in the sequence diagram

When the recipient requests access to data, the *AccessRequested* event is emitted along with the assetID of the data they wish to access. The data owner who subscribes to the *AccessRequested* event successfully listens to this event and fetches the recipient address using the *msg.sender* property which returns the address of the sender of the request, the recipient in this case. Then by using the mapping of addresses to public keys, retrieves the public key of the recipient. The data owner then generates a signer and verifier for future authentications. These authentications ensure that received kfrags or cfrags are from the correct data owner or proxy. The data owner then uses their secret key, the recipient public key, the signer, and a threshold t to generate the kfrags and uploads them and with the verifier to IPFS. After getting the corresponding CIDs, the data owner registers them on the blockchain, as parameters, using the *uploadedKfrag* smart contract function which emits the *KfragsAvailable* event that in turn passes as parameters the CIDs of the kfrags and the verifier to listeners. The threshold t , although not utilised, is still included to provide a easier extension point for future work. The recipient goes into an await state, listening for the *RecipientRetrieve* event.

4.2.3 Re-Encryption Phase

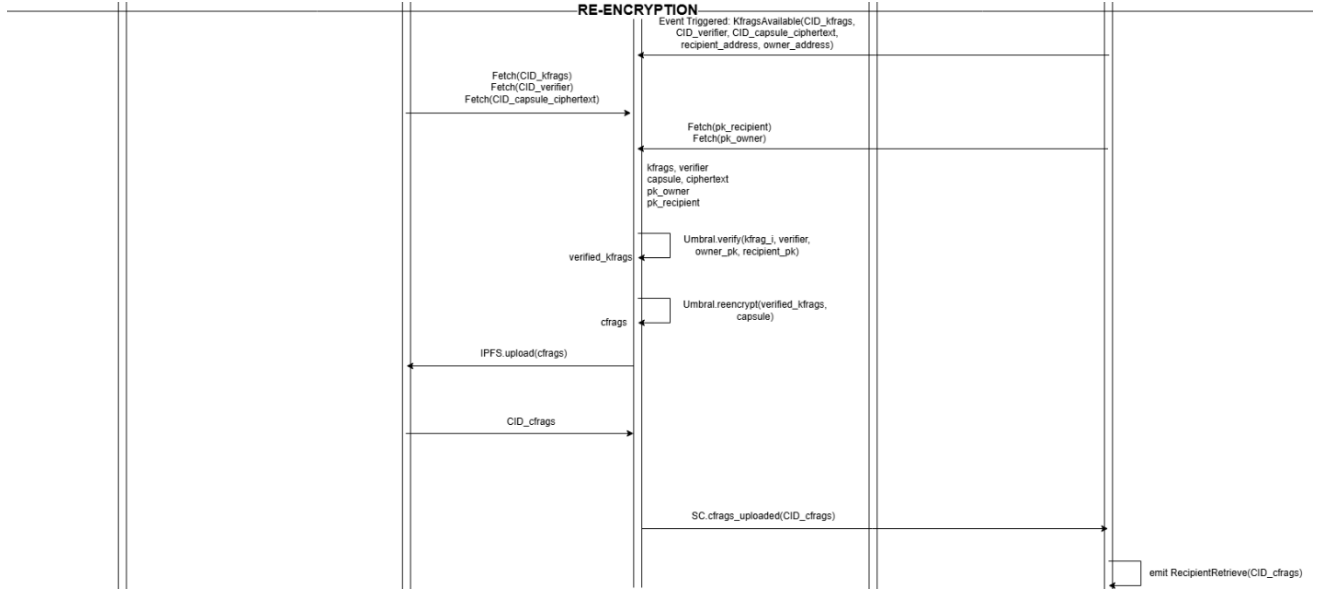


Figure 4.3: Screenshot of the re-encryption phase of the protocol in the sequence diagram

When the proxy successfully listens to the *KfragsAvailable* event and receives the CID of the kfrags, verifier, capsule and ciphertext, the recipient address, and the owner address, it fetches the corresponding data from IPFS using the CIDs and the public keys from the smart contract registry using the corresponding addresses. The proxy then verifies all the kfrags to get verified kfrags. Finally, it re-encrypts the verified kfrags using the *reencrypt* function to get cfrags. This re-encryption process is executed in a loop for each individual kfrag in kfrags and stops when at least t kfrags have been re-encrypted into cfrags. The cfrags are then uploaded to IPFS and the retrieved cfrags CID is then used as a parameter in the function *cfragsuploaded* which emits and passes the cfrags CID as a parameter, the *RecipientRetrieve* event.

4.2.4 Decryption Phase

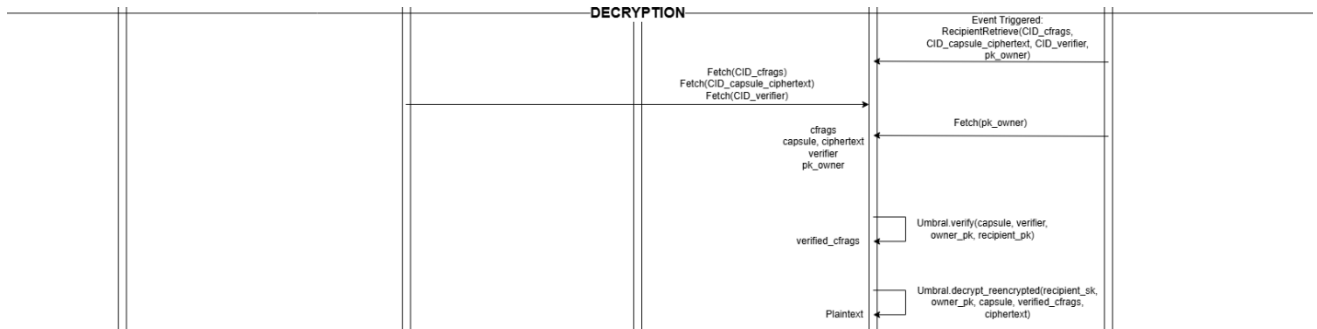


Figure 4.4: Screenshot of the decryption phase of the protocol in the sequence diagram

The recipient successfully listens to the *RecipientRetrieve* event and receives the CID of the cfrags, capsule and ciphertext, the verifier and the owner address. The recipient then fetches the

corresponding data from IPFs using the CIDs and the public key of the owner using the owner address. They then verify the cfrags to get verified cfrags. Finally, the recipient decrypts the ciphertext using the *decryptreencrypted* function that takes as parameters, the recipient private key, owner public key, capsule, verified cfrags and ciphertext.

4.3 Data Flow

The complete data flow of the protocol can be summarised in the four phases mentioned above:

- In Phase 1, All participants generate a keypair and registers their public keys on the blockchain. The owner encrypts the plaintext, and uploads the ciphertext and capsule to IPFS, registering the CID in the contract
- In Phase 2, The recipient requests to access the data and the owner generates the kfrags and uploads them to IPFS
- In Phase 3, The proxy fetches the required variables, verifies the kfrags, and computes cfrags
- In Phase 4, The recipient fetches all the required variables, verifies the cfrags, and decrypts the ciphertext

5 | Implementation

This chapter describes the technical decisions, process, and limitations identified while creating a functional prototype of the protocol design described in Chapter 4. It also specifies areas to explore in future works. The specific technologies used were: Python, the Umbral library, the Web3 library and Solidity.

5.1 Technology Stack

The core cryptographic operations of the data owner, recipient and proxy are implemented in Python using the Umbral library, which provides the functions required to develop an Umbral PRE protocol.

IPFS interactions use the ipfshttpclient library to communicate with a locally running IPFS daemon. Content is added and retrieved via the IPFS HTTP API. Uploaded content is pinned to ensure accessibility.

Smart contracts were written in Solidity as it is the most widely used language when drafting smart contracts for the Ethereum blockchain, and had extensive documentation.

Smart contract interactions like function calls, event subscriptions, and signing transactions were handled in Python using the Web3 library.

5.2 Technical Decisions

5.2.1 Segmentation of Scripts to Simulate Distributed Roles on a Single-Machine

A core challenge in implementing a PRE protocol is that it inherently involves multiple different parties, a data owner, a recipient and one or more proxy nodes. Each of these parties should be operating independently on separate machines with no direct access to one another's private keys. Ideally, in a production environment each role would be fulfilled by a different machine, however for this prototype, all roles were simulated on a single machine.

To simulate the separation of roles despite this constraint, the implementation was deliberately segmented into different scripts, each corresponding to a parties responsibilities. *UmbralEncryption.py* represents the owner, handling key generation, encryption and kfrag distribution. *zFetchData.py* represents the recipient, responsible for requesting acces and retrieving the encrypted payload. *UmbralDecryption.py* completes the recipient's role by performing decryption on the re-encrypted ciphertext.

This segmentation ensures that no single script has access to variables it would not actually possess in a real deployment. For example, the proxy never has access to either party's secret keys, only the essential variables required as denoted in the design. Communication between scripts is coordinated through a combination of on-chain events and shared JSON files, simulating the simultaneous, event driven nature of a real distributed system. For example, the owner's

script blocks and polls the blockchain for an *AccessRequested* event before proceeding with kfrag generation, meaning scripts must be run in separate terminals in a definite sequence, reinforcing the simulation of the multiple roles on one machine.

5.2.2 Supplementing JSON Files for IPFS Upload and Retrieval Calls

The decision to supplement some IPFS upload and retrieval functionality was made to simplify the process of obtaining of cryptographic material and to make debugging during development easier. JSON is human readable which helps tremendously during the prototyping phase as the contents of any information meant to be stored on IPFS can be inspected directly in my IDE. For the any present IPFS calls, the IPFS local daemon is accessed via its HTTP API at *http://127.0.0.1:5001*, with uploads performed via the add function and pinned via the pin function to ensure accessibility. Retrieval is performed through the gateway at *http://127.0.0.1:8080* using the Python requests library.

5.3 Smart Contract – Policy Registry Implementation

5.3.1 Policy Registry

The PolicyRegistry is the main coordination mechanism of the protocol and is the only component that all parties interact with. It enforces the correct ordering of operations without requiring any participant to trust another. The contract is deployed on a local Anvil testnet using the Foundry toolkit.

```
struct Asset {
    address owner;
    address recipient;
    bytes32 capsuleHash;
    string cid;
    bool active;
}

struct Account {
    address user;
    string publicKey;
}

uint256 public assetCount;
mapping(uint256 => Asset) public assets;

mapping(address => Account) public accounts;
```

Listing 5.1: Structs created in Solidity from the smart contract PolicyRegistry.sol

This contracts contains two main data structures, Asset and Account. The *Asset* struct contains the data the data owner wishes to share with another party and the relevant metadata. The *Account* struct contains a users Ethereum wallet address and public key. The mapping *mapping(uint256 => Asset) public assets* maps an unsigned integer which is the assetID, to an *Asset* struct, so everytime an owern registers a new file, an entry is created in this mapping with a unique increasing integer as the key. The mapping *mapping(address => Account) public accounts* maps an Ethereum wallet address to an *Account* struct. Every participant who registers on the contract gets an entry in this mapping keyed using their Ethereum address.

This contract contains the *requestAccess* function that represents the main communication layer of the implementation

```
function requestAccess(uint256 assetId) external {
    require(assets[assetId].active, "Asset not active");
    emit AccessRequested(assetId, msg.sender);
}
```

Listing 5.2: requestAccess function created in Solidity from the smart contract PolicyRegistry.sol

This function is called by the recipient when they wish to access a specific data. It passes as a parameter the *assetId* of the specific data and requires the asset to be active in order to pass the address of the recipient to the data owner, such that the data owner can then retrieve the recipients public key from the smart contract registry and compute the kfrags.

5.4 Proxy Functions

The proxy re-encryption step is implemented in *ProxyFunctions.py*, which is responsible for computing the cfrags by consuming the kfrags generated by the owner.



```
1  {
2    "cid": "QmXQkcNtY17i2dKYUBk1pywxjxeZQW73xg4stnX6uYPfiB",
3    "capsule_hash": "0x0f8f887966d8ed73a83fc8affe7f27031fde681d90a6f82decdbff41baef343d",
4    "owner_pk_b64": "A0xkv66H1Ujrb0ic8bkWb/zgQa9DH/5P0q/Ga7CdVmHN",
5    "recipient_sk_b64": "eB6o/veR7uZKzbxEt/8GbtZgTycZPjPdzH7zQ118NE=",
6    "recipient_pk_b64": "AtLh3QN3glkof+Cp012boJChF30n90MiLPcN/rMJM86o",
7    "verifier_pk_b64": "A0xkv66H1Ujrb0ic8bkWb/zgQa9DH/5P0q/Ga7CdVmHN",
8    "kfrags_b64": [
9      "gFaCxYsZ14eItWaIHPspm6DECbeKGVLotCwqI2J05XYorpuK5q+cv08qUET9hMDk+iMF/rnY5zemo8JNV3+m",
10     "1mlm3AwHvQ48PbpqWRWegoouqeAvtEzebpqvBKM4qoE/n75cttXwTdpoksatvztYXS1XB1jnQLbTNJS17hU",
11     "UsvjfazIkjGz4es5jilNHv0JenAbzUYzYZkC2reJx4XP1H7xM4nyGFF1CxMoVcCNvgYgCSUxf8ArVJ+QbNkd",
12     "dfpKU/kZaPAZJ5iTkLDQ5Bjc5cAKcCfUt+ZpHK9+hYbdd33ekn+MAUjC/nUPfafnv8uHuWgFLkXQu3v9K6r2",
13     "Y8jGcN8s16ZIEsYYPq6tX6W+EdmpV4XIECA2W0G6nM+asSW55+sZJaeWHAycLt/igoNobDlkuoWg8HMrjD",
14     "k6LAq1wi4q5gc/oJR6z9m8r64xxxAKpCj4yS6JR67LvUo/X2Y87wUrhmpokiyyiDiECILp6CoId4NSAzHgIE",
15     "mkE11dAoTDDuDIwe9L6CQOpsnKgBKILVFYwYeHw6NmqiHpXecgwoZucB3Qq+2jM7NE7sG0zn98fvjhbtX5m",
16     "PHXh32reCRGI+zdov/qEmYlqueFg/djPUHSPj86GcNCKBVGPivGhpfc+cdkZuAYWqZLHNGd1eVt5IBm9esxj",
17     "KcgYjs/y/UmYwQn3cNca0HXZz6RP70LAN1Pheue3kl0K4rZFImf/ubZFgf3Bs67FYd/KJaIr3SZXeu5M5L+8",
18     "JedvLgGR8RkBDjYM0rRhh91TRxYyzuggZgHz2C+W5vnB+A2dHDLcfqRRQlwS0upEZX5o1sT04HbaB0WYVo",
19   ],
20   "threshold": 5
21 }
```

Figure 5.1: Screenshot of the owner_state.json file

The script reads from *owner_state.json* which contains the owner's public key, the verifying key, and the serialised kfrags/ It also reads the encrypted metadata from *retrieved.json*, which contains the capsule retrieved from IPFS/ Each kfrag is deserialised and verified against the owner's verifier, the owner's public key, and the recipient's public key using the *verify* method provided by the Umbral library. This verification step confirms that the kfrags are legitimately generated by the owner for the intended recipient, and that they have not been tampered with.

Following verification, each verified kfrag is passed to the re-encrypt function along with the capsule to produce a cfrag.

```

verified_kfrags = []
for k in kfrags:
    vk = k.verify(verifying_pk, owner_pk, recipient_pk)
    verified_kfrags.append(vk)

cfrags = [reencrypt(capsule, vk) for vk in verified_kfrags[:threshold]]
t(f"Generated {len(cfrags)} cfrags.")

```

Listing 5.3: Verification and re-encryption process written in Python from the script *ProxyFunctions.py*

The cfrags are serialised to bytes and base64-encoded for storage and then written to *cfrags.json*, it also includes the asset ID and the CID of the ciphertext and capsule for use in the decryption process. In the current single proxy configuration, all kfrags are processed by a single script and the threshold parameter specifies the minimum number of cfrags required for decryption. The implementation generates a minimum threshold number of cfrags before passing them to the recipient.

5.5 Owner and Recipient Implementation

5.5.1 Data Owner

The owner's responsibilities are implemented in *UmbralEncryption.py*, which handles the key generation, encryption, on-chain registration, and kfrag generation. When ran, the script generates a random Umbral secret key for the owner and derives the corresponding public key. A recipient keypair is also generated in the same script for prototype purposes, in a deployment ready system, the recipient would generate their own keypair independently on their own machine.

Both public keys are then serialised to bytes and base64-encoded before being submitted to the *PolicyRegistry* contract via the *registerAccount* function. The plaintext is then encrypted using the owner's public key via the *encrypt* function from Umbral, producing a capsule and ciphertext. The capsule bytes are hashed using keccak to produce a capsule hash. The encrypted payload is then serialised to JSON and uploaded to IPFS, with the returned CID registered on-chain via *registerAsset*.

After asset registration, the owner's script enters a polling loop, querying the *PolicyRegistry* contract for *AccessRequested* events with the registered asset ID every 2 seconds. When such an event is detected, the script retrieves the recipient's public key from the contract using their Ethereum wallet address, uses it to generate kfrags via *generate_kfrags*, and writes them into *owner_state.json* for use by the proxy.

5.5.2 Recipient

The recipient's main responsibilities is implemented in *zFetchData.py*, which submits a *requestAccess* transaction to the *PolicyRegistry* contract to request access and trigger the kfrag generation process from the owner.

```

tx = contract.functions.requestAccess(asset_id).build_transaction({
    "from": recipient_account.address,
    "nonce": w3.eth.get_transaction_count(recipient_account.address),
    "gas": contract.functions.requestAccess(asset_id).estimate_gas({"from":
        recipient_account.address}),

```

```

    "gasPrice": w3.eth.gas_price,
  })
  signed = recipient_account.sign_transaction(tx)
  tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
  w3.eth.wait_for_transaction_receipt(tx_hash)

```

Listing 5.4: Construction of the *requestAccess* transaction written in Python from the script *zFetchData.py*

The above lines builds the *requestAccess* transaction *tx* from scratch and sends it onto the blockchain using a Web3 RCP call.

The script then reads the assets IPFS CID from the contract and fetches the encrypted payload from IPFS, saving the ciphertext and capsule to *retrieved.json* for use in the final decryption process. Decryption is performed in *UmbralDecryption.py*, which takes as parameters, the cfrags from *cfrags.json* and the encrypted payload from *retrieved.json*, verifies each cfrag, and finally calls *decrypt_reencrypted* to recover the original plaintext

5.6 Implementation Limitations

5.6.1 Unencrypted kfrag Security Limitation

A significant security limitation of the current implementation is that kfrags are stored in plaintext within *owner_state.json* and passed to the proxy without any encryption or access control. This problem persists within the design of the system as well. In a deployment ready PRE system, kfrags should be encrypted to the public key of their intended proxy node before distribution, such that only the designated proxy can decrypt and use its assigned fragment. This ensures that even if the distribution of kfrags is compromised, an attacker cannot reconstruct the delegating secret key by collecting a threshold number of fragments.

In this prototype and design, the absence of kfrag encryption means that an attacker could extract all the kfrags and perform re-encryption without authorisation. This is an acceptable gap in a prototype where the focus is on demonstrating the cryptographic correctness of the proposed protocol, but it is a real critical vulnerability that would need to be addressed in real-world deployment.

5.6.2 Blockchain Pseudonymity Security Limitation

The use of a public blockchain as the coordination layer introduces pseudonymity limitation. While Ethereum addresses are not directly linked to real-world identities, all transactions and event emitted by the *PolicyRegistry* contract are publicly visible on-chain, including *AssetRegistered* events that reveal the owner and recipient addresses, and *AccessRequests* events that reveal when and who wishes to access a piece of data.

In a threat model where an adversary is able to link Ethereum addresses to real-world identities, the on-chain ledgers becomes a public record of who accessed what and when they did so. This is a fundamental contradiction between the transparency and audit-ability properties of public blockchains and the privacy requirements of a data access control system.

6 | Conclusion

6.1 Summary

This dissertation detailed the design and implementation of an decentralised file transfer protocol combining Umbral proxy re-encryption, Ethereum smart contracts, and IPFS. The goal was to demonstrate that a data owner can transfer data to a designated recipient without any intermediary gaining access to the plaintext, and without relying on a central authority for governance and coordination. A working prototype was implemented and evaluated, demonstrating correct end-to-end operation. While the prototype does make deliberate simplifications, such that adopting a single-proxy configuration, unencrypted key fragments, and supplementing IPFS interactions, the core cryptographic and coordination mechanisms function as designed and should serve as an interesting basis for further research in the subject area.

A Appendices

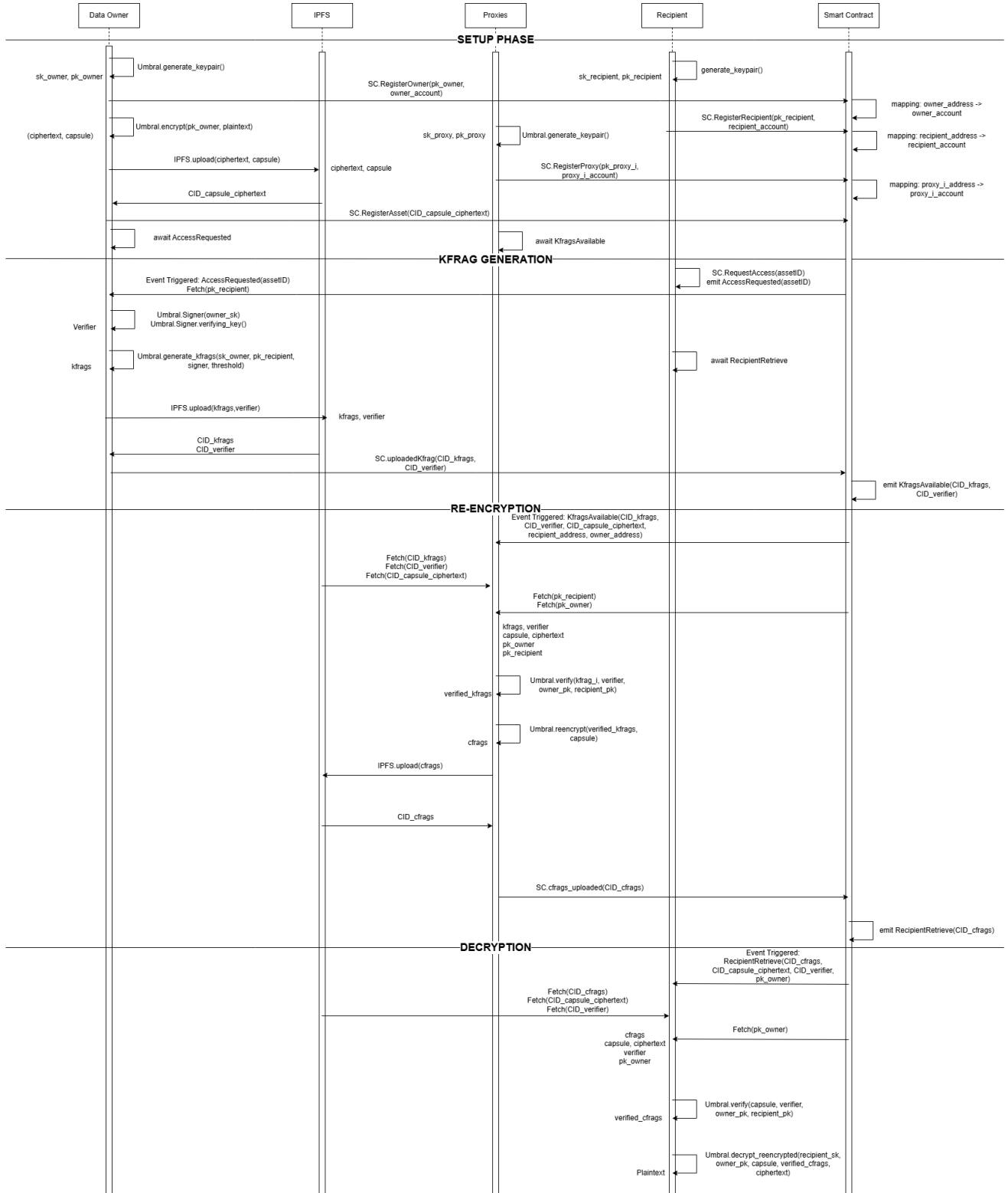


Figure A.1: Sequence Diagram of Protocol

Bibliography

Núñez, D., Agudo, I. and Lopez, J. (2017) 'Proxy re-encryption: Analysis of constructions and its application to secure access delegation', *Journal of Network and Computer Applications*, 87, pp. 193–209. doi:10.1016/j.jnca.2017.03.005.

De Filippi, P., Wray, C. and Sileno, G. (2021) 'Smart contracts', *Internet Policy Review*, 10(2). doi:10.14763/2021.2.1549.

Shi, R., Cheng, R., Han, B., Cheng, Y. and Chen, S. (2024) 'A closer look into IPFS: Accessibility, content, and performance', *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(2), Article 20. Available at: <https://doi.org/10.1145/3656015>

Wang, S. et al. (2025) Time tells all: Deanonymization of blockchain RPC users with zero transaction fee (extended version). arXiv preprint arXiv:2508.21440. Available at: <https://arxiv.org/abs/2508.21440>

Protocol Labs (2024) Content addressing and CIDs, IPFS Documentation. Available at: <https://docs.ipfs.tech/concepts/content-addressing/#what-is-a-cid>

Muthe, K.B., Vemuru, T.S.T., Sharma, K. and Mohammad, N.S. (2020) 'Decentrinet: An Ethereum, proxy re-encryption and IPFS based decentralized internet', 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), pp. 1–5. doi: 10.1109/ICCCNT49239.2020.9225483.

Manzoor, A., Braeken, A., Kanhere, S.S., Ylianttila, M. and Liyanage, M. (2021) 'Proxy re-encryption enabled secure and anonymous IoT data sharing platform based on blockchain', *Journal of Network and Computer Applications*, 176, p. 102917. doi: 10.1016/j.jnca.2020.102917.